```
if(usb_route[addr].hub)                          // 在 Hub 后面：split 事务
  split = HCSPLT_SPLTENA | HCSPLT_XACTPOS_ALL |
          HCSPLT_HUBADDR(usb_route[addr].hub) | HCSPLT_PRTADDR(usb_route[addr].port) |
          (req->complete_split ? HCSPLT_COMPSPLT : 0);
wr(HCSPLT(6), split);
wr(HCDMA(6),  DWC2_DMA_BUS(urb->transfer_buffer));
wr(HCTSIZ(6), HCTSIZ_XFERSIZE(8) | HCTSIZ_PKTCNT(1) | HCTSIZ_PID(pid));
wr(HAINTMSK, rd(HAINTMSK) | (1U << 6));
wr(HCCHAR(6), HCCHAR_DEVADDR(addr) | HCCHAR_EPNUM(ep) | HCCHAR_EPTYPE(EPTYPE_INTERRUPT) |
              HCCHAR_MPS(8) | HCCHAR_EPDIR_IN |
              (usb_route[addr].low_speed ? HCCHAR_LSPDDEV : 0) | HCCHAR_CHENA);
req->last_issue_frame = rd(HFNUM) & 0x3fff;     // 记下发出时的微帧号
```

把一次按键的完整所有权转移摊开，是这样一条链：

```
usbkbd_probe()
  -> usb_alloc_urb() / usb_fill_int_urb(EP1 IN, 8 字节, interval 10 ms)
  -> usb_submit_urb(urb)              URB 标记为 submitted，所有权交给 HCD
  -> udev->ops->submit_urb()          DWC2 配置 ch6 DMA 后立即返回
  -> HCD 推进 SSPLIT/CSPLIT，有报告时 DMA 写入 buffer
  -> usb_hcd_giveback_urb(status, actual_length)   所有权交还类驱动
  -> urb->complete(urb)               usbkbd_irq_complete()
  -> queue_work(&usbkbd_wq, &kbd->rx_work)   用 urb->context 找回这台键盘
  -> usbkbd_rx_work() 解析报告，最后再次 usb_submit_urb()
```

每设备状态，不是全局单例. usbkbd_state 由 usbkbd_probe() 用 kalloc() 逐设备分配，保存在 udev->dev.driver_data，报告缓冲、上一份报告、capslock 状态、rx_work、自己的 kbd->lock，以及那只 interrupt-IN URB 全在这份私有状态里。完成回调只认 urb->context，不再依赖一个静态的“当前键盘”。输入设备名 USB HID Boot Keyboard、phys 为 usb/dwc2/input0、bustype、从描述符读到的 VID/PID 也在这里填好，再用 input_set_capability() 把 hid_to_key[] 里所有非零项登记成这台设备支持的按键能力位图。传输、完成回调、work 和生命周期因此都是按设备走的：换一只键盘、再插一只鼠标，用的是同一段类驱动代码，而不是某个被硬编码的设备。

之后的事全由中断推动（图 7.9）。dwc2_irq() 先交给快速路径 dwc2_interrupt_rx_irq_fast()：SSPLIT 的 ACK 和窗口内的 NYET 在这里当场处理——记下当前微帧，忙等到“发出后第 2 个微帧”（最多 500 μs），立即重发 CSPLIT——因为它们有微帧级的截止时间，交给任何可调度的线程都来不及。只有最终结果（完成、无数据或错误）才屏蔽 ch6 并 schedule_work。HCD 的完成 work 做缓存维护、算实际长度并归还 URB；键盘的完成回调只 queue_work(kbd->rx_work)；usbkbd_rx_work() 在进程上下文里比较新旧报告、上报输入事件、调用 consoleintr()，最后重新 usb_submit_urb()。硬中断只管微帧时序，公共 system_wq 只判定控制器结果，专用 periodic workqueue 只负责到期重试，类驱动自己的 usbkbd_wq 只管 HID。

NAK 的无忙等周期重试. ch6 的最终 IRQ 仍通过 schedule_work(&interrupt_rx.work) 立即进入公共 system_wq。dwc2_interrupt_urb_work() 调用 dwc2_interrupt_rx_complete() 读取并清除 HCINT；若本轮是 NAK，或 CSPLIT 已越过当前 frame 的有效窗口，它不再执行毫秒级 udelay(bInterval)，而是调用：

```
mod_delayed_work(&dwc2_periodic_wq, &interrupt_rx.retry_work,
                 interval_jiffies);
```

这时 retry_work 先进入 workqueue 核心按 expires 排序的全局 delayed_head，并未进入可执行队列。CPU0